

1 Поиск подстроки в строке

1.1 Домашнее задание

1. Для каждого префикса строки найти количество его префиксов, равных его суффиксу. $\mathcal{O}(n)$.
2. За $\mathcal{O}(n)$ построить строку с данной префикс функцией.
3. Преобразовать Z-функцию в префикс-функцию без промежуточного восстановления строки за $\mathcal{O}(n)$.
4. Дана строка s , нужно за $\mathcal{O}(|s|)$ найти такую минимальную строку t , что st – палиндром.
5. Дан набор n непустых строк s_i . Для каждой s_i найдите $\min$ по длине префикс, который не является префиксом других строк. $\mathcal{O}(\sum |s_i|)$.

Дополнительные задачи

6. За одну секунду в конец изначально пустого текста дописывается случайная буква (равномерное распределение). Чему равно матожидание времени T , когда первый раз s станет подстрокой выписанного текста?

1.2 Решения

1. (a) Вычисление префикс-функции:

- Для строки s длины n вычисляем массив $p[0..n-1]$, где $p[i]$ — длина наибольшего собственного префикса подстроки $s[0..i]$, совпадающего с её суффиксом.

(b) Построение массива $count$:

- Создаём массив $count[0..n-1]$, где $count[i]$ — количество нетривиальных совпадений префиксов и суффиксов для $s[0..i]$
- Правило заполнения:

$$count[i] = \begin{cases} 1 + count[p[i] - 1], & \text{если } p[i] > 0 \\ 0, & \text{иначе} \end{cases}$$

(c) Формирование ответа:

- Для префикса длины $i+1$: ответ = $count[i] + 1$
(добавляется тривиальное совпадение полной строки с собой)

2. Будем строить строку слева направо. Для начала присваиваем первому символу букву 'a'.

Далее проходим по всей префикс-функции. Если в текущей позиции значение равно нулю, это означает, что ни один префикс не совпадает с суффиксом текущего префикса. Значит, необходимо добавить новый символ, ранее не встречавшийся в активных префиксах. Для этого просто берём следующую по порядку латинскую букву.

Если же значение префикс-функции в позиции отлично от нуля, это означает, что текущий символ должен совпадать с символом, стоящим на позиции, равной значению префикс-функции минус один. Таким образом, мы просто копируем нужный символ из ранее построенной части строки.

Поскольку значения префикс-функции могут увеличиваться максимум на один, корректное соответствие символов гарантировано. Если в процессе закончились латинские буквы (что возможно только при очень длинной последовательности с нулями), можно использовать любую букву, не совпадающую с первой буквой строки и символом, который должен был бы идти по текущему префиксу, чтобы сохранить корректность.

3. Пусть Z-функция хранится в массиве $z[0..n-1]$, а префикс-функция — в массиве $p[0..n-1]$.

Заметим, что если $z[i] > 0$, то для всех индексов $i+j$, где $0 \leq j < z[i]$, значение

$$p[i+j] \geq j+1,$$

поскольку подстрока $s[i..i+j]$ совпадает с префиксом $s[0..j]$.

Также отметим следующее: если значение $p[i+j]$ уже установлено при проходе с позиции i , и позже мы пытаемся установить значение $p[i'+j']$, где $i < i'$ и $i+j = i'+j'$, то новое значение будет не больше уже установленного, то есть

$$j+1 > j'+1,$$

следовательно, обновлять его не имеет смысла, так как префикс-функция должна хранить максимальное значение совпадения.

Отсюда вытекает алгоритм:

- Идём слева направо по массиву z .
- Для каждого i , если $z[i] > 0$, проходим в обратном порядке по индексам $j = z[i] - 1, z[i] - 2, \dots, 0$ и пытаемся установить

$$p[i+j] := j+1,$$

пока не наткнёмся на уже инициализированный элемент $p[i+j] \neq 0$.

- При достижении первого уже заполненного элемента прерываем внутренний цикл, так как слева от него значения обновлять не нужно.

Каждый элемент массива p устанавливается ровно один раз. После этого при попытке повторного установления прерываем внутренний цикл. Таким образом, количество операций установки равно n , а количество срабатываний `break` — не больше n , так как каждый `break` переводит внешний цикл к следующей итерации.

4. (a) Построим строку $a = s + \# + \text{reversed}(s)$, где $\#$ — символ, не встречающийся в s .
 (b) Вычислим префикс-функцию p для строки a .
 (c) Последнее значение $p[-1]$ показывает длину наибольшего префикса строки $\text{reversed}(s)$, совпадающего с суффиксом s .
 (d) Тогда строка $t = \text{reversed}(s)[p[-1] :]$ — минимальное дополнение к s , чтобы получить палиндром.

5. Построим бор из всех строк множества S . В каждой вершине бора будем хранить бул `is_single`, означающий, что соответствующий префикс встречается ровно в одной строке.

При добавлении каждой строки в бор для каждого символа проверяем, существует ли переход по этому символу. Если переход существует, устанавливаем `is_single` в значение `false`, так как данный префикс теперь встречается как минимум в двух строках. Если переход создаётся впервые, флаг устанавливаем в `true`.

После добавления всех строк для каждой строки s_i пройдем по её символам в боре и найдём первый префикс, вершина которого имеет `is_single = true`. Этот префикс и будет минимальным уникальным префиксом для s_i .

Если такой префикс не найден, то минимальным уникальным префиксом считается вся строка s_i .

6. ...